Question

Case Study on Class & Objects (CO1): Bank Account Management System Design a system to manage bank accounts using Object Oriented Programming concepts.

- **Classes:** Account, SavingsAccount, CurrentAccount
- **Attributes:** Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account)
- **Methods:** Deposit, Withdraw, Calculate Interest, Transfer
- **Object Usage:** Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

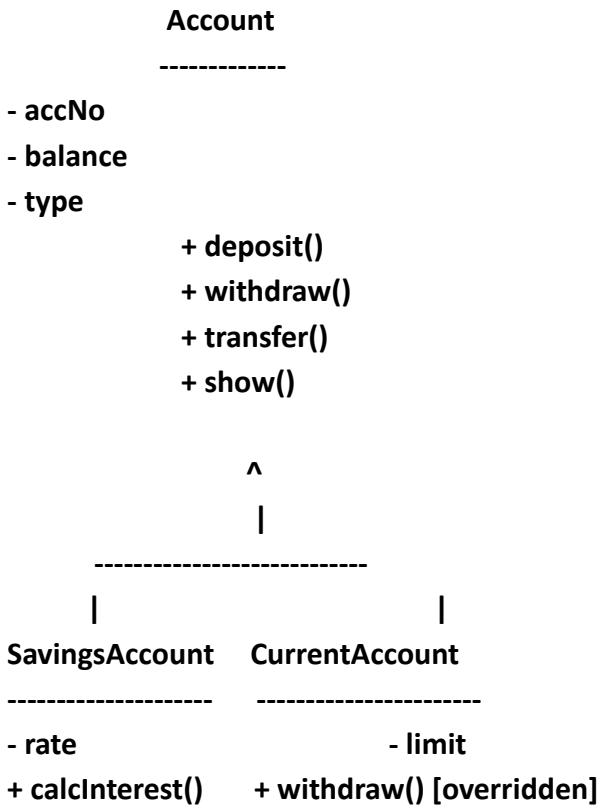
Steps for Execution

1. Install **Java JDK** (version 17 or above).
2. Open any IDE (Eclipse, IntelliJ, VS Code) or use command line.
3. Save the program in a file named BankSystem.java inside package javacore.
4. Compile the program: javac BankSystem.java
5. Run the program: `java BankSystem`
6. Use the menu-driven options to perform deposit, withdraw, interest calculation, transfer, and display account details.

Algorithm

1. Define a **base class** Account with attributes: accNo, balance, type.
2. Implement methods: deposit(), withdraw(), transfer(), show().
3. Create a **derived class** SavingsAccount with attribute rate and method calcInterest().
4. Create another **derived class** CurrentAccount with attribute limit and overridden method withdraw() (supports overdraft).
5. In the **main class** BankSystem, create objects of SavingsAccount and CurrentAccount.
6. Provide a **menu-driven interface** using Scanner for user input.
7. Perform operations based on user choice.
8. Exit when the user chooses option 7.

UML Class Diagram:



CODE:

```
package javacore;

import java.util.Scanner;

//Base class for all accounts
class Account {
 int accNo;       // account number
 double balance;   // money in account
 String type;      // savings or current

 Account(int no, double bal, String t) {
    accNo = no;
```

```
    balance = bal;
    type = t;
}

// deposit money
void deposit(double amt) {
    balance += amt;
    System.out.println("Deposited: " + amt);
    System.out.println("Balance: " + balance);
}

// withdraw money
void withdraw(double amt) {
    if (amt <= balance) {
        balance -= amt;
        System.out.println("Withdrawn: " + amt);
        System.out.println("Balance: " + balance);
    } else {
        System.out.println("Not enough balance!");
    }
}

// transfer money to another account
void transfer(Account other, double amt) {
    if (amt <= balance) {
        balance -= amt;
        other.balance += amt;
        System.out.println("Transfer done!");
```

```
    } else {
        System.out.println("Transfer failed!");
    }
 }

 // show account info
 void show() {
    System.out.println("Acc No: " + accNo);
    System.out.println("Type  : " + type);
    System.out.println("Bal   : " + balance);
 }
}

//Savings account class
class SavingsAccount extends Account {
 double rate; // interest rate

 SavingsAccount(int no, double bal, double r) {
    super(no, bal, "Savings");
    rate = r;
 }

 void calcInterest() {
    double interest = balance * rate / 100;
    System.out.println("Interest: " + interest);
 }
}
```

```
//Current account class
class CurrentAccount extends Account {
 double limit; // overdraft limit

 CurrentAccount(int no, double bal, double l) {
    super(no, bal, "Current");
    limit = l;
 }

 // overriding withdraw
 @Override
 void withdraw(double amt) {
    if (amt <= balance + limit) {
       balance -= amt;
       System.out.println("Withdraw ok. Balance: " + balance);
    } else {
       System.out.println("Overdraft limit crossed!");
    }
 }
}

//main class
public class BankSystem {
 public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    SavingsAccount sAcc = new SavingsAccount(101, 5000, 5);
    CurrentAccount cAcc = new CurrentAccount(201, 3000, 2000);
```

```
int ch;
do {
   System.out.println("\n--- Bank Menu ---");
   System.out.println("1. Deposit (Savings)");
   System.out.println("2. Withdraw (Savings)");
   System.out.println("3. Interest (Savings)");
   System.out.println("4. Withdraw (Current)");
   System.out.println("5. Transfer Savings -> Current");
   System.out.println("6. Show Accounts");
   System.out.println("7. Exit");
   System.out.print("Choice: ");
   ch = sc.nextInt();

   switch (ch) {
      case 1:
         System.out.print("Amount: ");
         sAcc.deposit(sc.nextDouble());
         break;
      case 2:
         System.out.print("Amount: ");
         sAcc.withdraw(sc.nextDouble());
         break;
      case 3:
         sAcc.calcInterest();
         break;
      case 4:
         System.out.print("Amount: ");
```

```
                cAcc.withdraw(sc.nextDouble());
                break;
            case 5:
                System.out.print("Amount: ");
                sAcc.transfer(cAcc, sc.nextDouble());
                break;
            case 6:
                System.out.println("Savings:");
                sAcc.show();
                System.out.println("Current:");
                cAcc.show();
                break;
            case 7:
                System.out.println("Bye!");
                break;
            default:
                System.out.println("Wrong choice!");
        }
    } while (ch != 7);

    sc.close();
 }
}
```

Output Screenshot (Sample Run)

--- Bank Menu ---

1. Deposit (Savings)

2. Withdraw (Savings) 3. Interest (Savings)

4. Withdraw (Current)

5. Transfer Savings -> Current

6. Show Accounts

7. Exit

Choice: 1

Amount: 2000

Deposited: 2000.0

Balance: 7000.0

Choice: 3

Interest: 350.0

Choice: 4

Amount: 4000

Withdraw ok. Balance: -1000.0

Choice: 5

Amount: 1000

Transfer done!

Choice: 6 Savings:

Acc No: 101

Type : Savings

Bal : 6000.0 Current:

Acc No: 201

Type : Current

Bal : 4000.0

Choice: 7 Bye!

//END